Author: Micheal Lanham

SECTION 1

Understanding Large Language Models (LLMs)

- **Probabilistic Token Machines:** LLMs generate text by predicting the most probable next token based on training distributions.
- **Tokens & Context Budgeting:** Text length does not equal token length; tracking token counts (via tools like `tiktoken` or SDK telemetry) is necessary to keep interactions within context windows and control costs.
- **Generation Knobs & Parameters:**
- **Temperature:** Controls response variability. Setting `temperature = 0` forces the model to select the highest-probability token at each step, ensuring consistent, repeatable output for debugging and planning.
- **Top-P:** An alternative sampling method based on cumulative probability thresholds. Providers recommend adjusting either `temperature` or `top-p`, but not both simultaneously.
- **Max Tokens & Seed:** `max_tokens` limits response length to prevent runaway API costs, while `seed` enables reproducible output during evaluation runs.

SECTION 2

Controlling LLMs with Prompt Engineering (Agent Persona)

- **Defining the Persona Layer:** Prompt engineering establishes the agent's system prompt (its persona layer), defining its role, tone, domain focus, and operating constraints.
- **Core Prompting Techniques:** Effective agent prompts rely on assigning a clear role, front-loading primary instructions, fencing user inputs with unique delimiters, providing few-shot examples, and incorporating step-by-step reasoning.
- **Avoiding Prompt Pitfalls:** Over-engineering prompts can cause failures due to excessive complexity, contradictory instructions, inconsistent delimiters, or rule-stuffing. Complex workflows should be broken down into smaller, role-specific agent steps rather than packed into a single monolithic prompt.

SECTION 3

Building Agents with the OpenAI Agents SDK

- **Minimal Agent Setup:** Demonstrates how to configure an agent's name, system instructions, and model settings, then execute runs synchronously or asynchronously via a runner.
- **Typed Inputs and Outputs:** Uses Pydantic schemas to strongly type agent responses (`output_type`). Strongly typed I/O removes fragile string parsing and prevents response variability from breaking downstream agent steps.
- **Agent Tracing:** Leverages built-in SDK tracing and execution logs to inspect prompt calls, token overhead, and step-by-step decision paths for debugging and optimization.

SECTION 4

Enhancing Agents through Tool Integration

- **From Prompt Chaining to True Agency:** Simple prompt chaining passes static text between prompts, whereas true agency requires providing tools (local functions, APIs, or MCP servers) that the agent can autonomously choose when and how to execute.
- **Tool Chaining & Scoping:** Agents sequence multiple tool calls autonomously to complete higher-level goals. Keeping tool lists lean and scoped reduces context overhead and minimizes tool selection errors.